Server Uptime Monitoring Log

Python Project Documentation

Semester 3 | Section J | Group 1

Group members

Kazi Mustafijur Rhaman | Anisha Akhtar
Koyel Basu | Suraiya Yeasmin

Abstract

Server Uptime Monitoring Log, presented through the Pulse dashboard, is a Python project that simulates one server ping reading per day. A for loop generates and processes readings, while conditions classify each day as healthy, minor lag or failure. The continue statement bypasses healthy processing after an ordinary lag spike. The break statement halts monitoring after a critical response or repeated lag. The project includes a Tkinter interface, genuine step-by-step execution, configurable thresholds, charts, saved-run comparison and text or CSV export. Two fixed demonstrations verify the stopping rules. The result is an explainable simulation for learning Python control flow rather than a measurement of a real server.

Introduction and problem statement

A monitoring log should distinguish a temporary delay from a condition serious enough to stop further checks. Our project models this decision using generated response times. It records why each checked day was classified, preserves the results and clearly identifies days that were not evaluated after a failure. This makes loop behaviour visible during a classroom demonstration.

Objectives

- Use a for loop and random integers to simulate daily ping statuses.
- Apply continue to skip normal healthy-day processing after minor lag.
- Apply break to stop on immediate failure or the configured lag streak.
- Explain, validate, compare and export the performed checks through a usable interface.

Software requirements

Python 3.10 or later, Tkinter and a graphical desktop are required for the dashboard. The terminal version uses the same core logic. The application relies on the Python standard library, including random, csv, json and unittest; no third-party pip packages are required.

Design and algorithm

The interface in gui.py passes the selected days, scenario, seed and limits to iter_monitor() in main.py. This shared generator processes one reading when advanced. The terminal consumes it automatically; teaching mode advances it through Next Day. Both paths use the same decisions and save the final report.

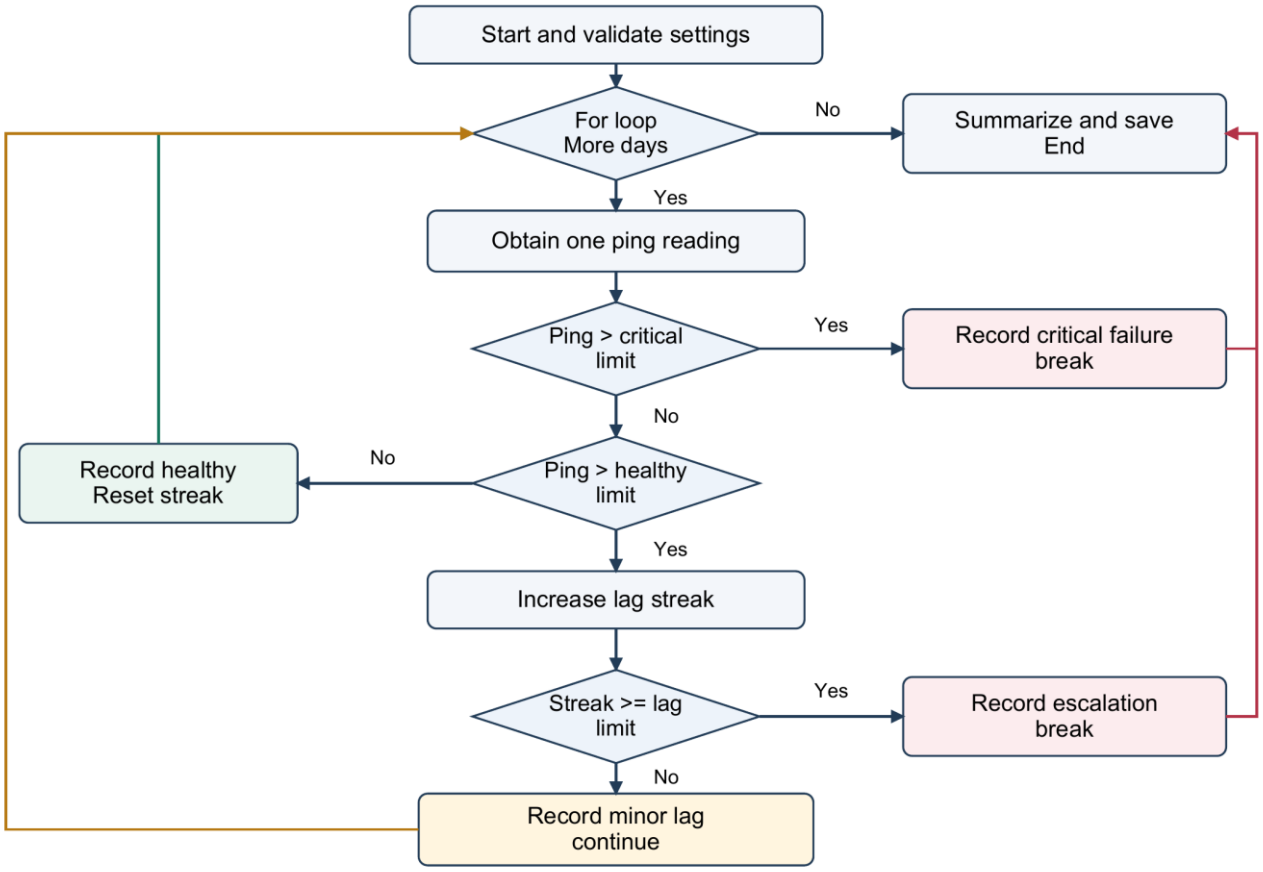


Figure 1 Decision flow of the shared Python simulation. Both continue and a healthy day return to the for loop; break leads to the final summary.

Default decision rules

Condition	Recorded result	Loop behaviour
0 to 100 ms	Healthy; reset streak	Record the day
101 to 300 ms before streak limit	Minor lag; increase streak	continue
Third consecutive lag reading	Escalated failure	break
Above 300 ms	Critical failure	break

Critical ping is checked first because it also exceeds the healthy limit. Custom settings require a non-negative healthy limit below the critical limit, and a lag limit of at least 1. The escalation day counts as a failure, not also as minor lag. Control statements execute before the entry is presented, and no later ping is consumed after failure.

Implementation and graphical results

Stable, busy and unstable scenarios generate integers in the inclusive ranges 20-140, 60-320 and 80-450 ms. A seed makes a random run repeatable. Fixed demos instead use predefined readings. Teaching mode highlights for, continue or break, while the dashboard updates the daily table, average ping, highest ping and longest lag streak. Completed runs are saved as JSON and text; History compares two records, and CSV export includes only checked rows and their rule settings.

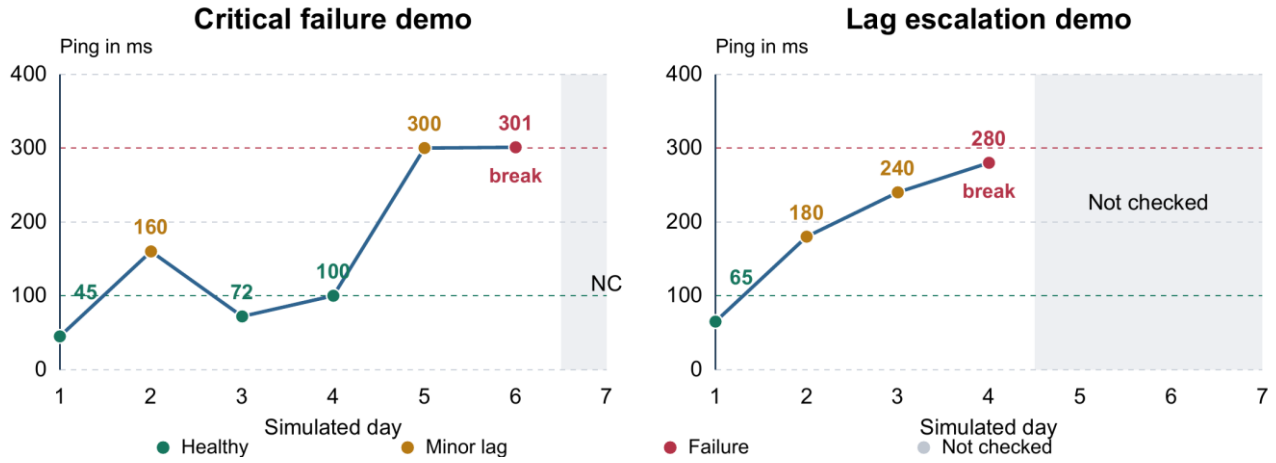


Figure 2 Demo ping traces under the default rules. Grey areas are not checked; NC means not checked. Dashed lines at 100 and 300 ms show the healthy and critical limits.

Metric	Critical failure demo	Lag escalation demo
Checked days of 7	6	4
Healthy / minor lag / failure	3 / 2 / 1	1 / 2 / 1
Days not checked	1	3
Average ping	163.00 ms	191.25 ms
Highest ping	301 ms	280 ms
Longest lag streak	1 day	3 days
Healthy check rate	50%	25%

The first demo stops on day 6 when 301 ms exceeds the critical limit. The second stops on day 4 at 280 ms: although it is below 300 ms, it is the third slow day in a row. These different stop reasons demonstrate both failure rules.

Average ping = sum of checked ping values / checked days.
Healthy check rate = healthy days / checked days x 100.

For example, the first demo averages (45 + 160 + 72 + 100 + 300 + 301) / 6 = 163 ms. Its seventh reading is excluded because the loop has already stopped. These sample statistics do not measure real-time service uptime.

Testing and validation

The project contains 50 automated tests: 39 core, command-line and persistence tests, plus 11 desktop integration tests. Core checks passed during report preparation. The corresponding GitHub workflow also passed its Windows desktop job. The following cases summarize the expected behaviour.

Test case	Expected behaviour
100 ms and 101 ms	Healthy at 100; minor lag and continue at 101.
300 ms and 301 ms	Lag at 300 unless streak limit is reached; immediate failure at 301.
Three consecutive lag readings	Escalation on the third; no following reading consumed.
Healthy day between lag sequences	Consecutive-lag streak resets.
Next Day in teaching mode	Exactly one reading per press; final step completes the run.
Invalid thresholds or days	Clear validation error before a run is started.
CSV and historical records	Export only checked rows; retain settings and read older valid logs.

How to run and demonstrate

Open the project folder in VS Code, choose Terminal then New Terminal, and run .\run_gui.bat on Windows, or python gui.py with Python installed. Select Critical failure demo and click Run simulation. Enable Step-by-step teaching before a new run, then use Next Day to explain each action. Use Thresholds to change the rules, History with Ctrl+click to compare two runs, and Export CSV to save the readings.

Conclusion

The project meets its objective by generating a readable server-health log and using for, continue and break to control the checks. The shared generator keeps the terminal and dashboard consistent. Fixed demos, checked-day statistics and automated tests make the behaviour reproducible and suitable for explaining Python fundamentals in a Semester 3 presentation.

Limitations and future scope

Readings are simulated; no actual server is contacted and no real days pass. High ping is treated as failure only by the chosen teaching rules. Future work could add real network measurements, timestamps, scheduled checks, retry and recovery handling, and optional notifications after a failure.

References

Project source and README: github.com/k-m-rahaman/Server_Uptime_Monitoring_Log

Python documentation: docs.python.org/3/tutorial/controlflow.html and docs.python.org/3/library/random.html